```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score,
    roc_curve, f1_score, precision_score, recall_score,
    accuracy_score
)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
plt.style.use('seaborn-v0_8-darkgrid')

print("Libraries imported successfully")
```

```
Libraries imported successfully
```

```
# --- File Upload Boxes for Colab ---
from google.colab import files
import pandas as pd

print("Upload TRAIN file (Ad_click_prediction_train...)")
uploaded_train = files.upload()

# Extract uploaded train file name dynamically
TRAIN_PATH = list(uploaded_train.keys())[0]
print("TRAIN file uploaded as:", TRAIN_PATH)

print("\nUpload TEST file (Ad_Click_prediciton_test...)")
uploaded_test = files.upload()

# Extract uploaded test file name dynamically
TEST_PATH = list(uploaded_test.keys())[0]
print("TEST file uploaded as:", TEST_PATH)

# Now load them safely
train_df = pd.read_csv(TRAIN_PATH)
test_df = pd.read_csv(TEST_PATH)

print("\nFiles loaded successfully!")
print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
```

```
Upload TRAIN file (Ad_click_prediction_train...)
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Ad_click_prediction_train (1).csv to Ad_click_prediction_train (1) (1).csv
✓ TRAIN file uploaded as: Ad_click_prediction_train (1) (1).csv

Upload TEST file (Ad_Click_prediciton_test...)
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Ad_Click_prediciton_test.csv to Ad_Click_prediciton_test.csv
✓ TEST file uploaded as: Ad_Click_prediciton_test.csv

🎉 Files loaded successfully!
Train shape: (463291, 15)
```

```
train_df = pd.read_csv(TRAIN_PATH)
test_df = pd.read_csv(TEST_PATH)

# Align with the rest of the pipeline
train_df = train_df.rename(columns={
    "DateTime": "timestamp",
```

```
    "is_click": "click"
})
test_df = test_df.rename(columns={
    "DateTime": "timestamp"
})

print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
train_df.head()
```

```
Train shape: (463291, 15)
Test shape: (128858, 14)
```

	session_id	timestamp	user_id	product	campaign_id	webpage_id	product_category_1	product_category_2	user_group_id
0	140690	2017-07-02 00:00	858557	C	359520	13787	4	NaN	10.0
1	333291	2017-07-02 00:00	243253	C	105960	11085	5	NaN	8.0
2	129781	2017-07-02 00:00	243253	C	359520	13787	4	NaN	8.0

```
print("\n=== DATA INFO (TRAIN) ===")
print(train_df.info())

print("\n=== DESCRIPTIVE STATS (NUMERIC) ===")
display(train_df.describe())

print("\n=== MISSING VALUES (TRAIN) ===")
missing = train_df.isnull().sum()
missing_pct = (missing / len(train_df)) * 100
missing_df = pd.DataFrame({"Missing_Count": missing, "Percentage": missing_pct})
display(missing_df[missing_df["Missing_Count"] > 0])

print("\n=== TARGET DISTRIBUTION (click) ===")
print(train_df["click"].value_counts())
ctr = train_df["click"].mean() * 100
print(f"\nOverall CTR: {ctr:.2f}%")

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
train_df["click"].value_counts().plot(kind="bar", ax=axes[0])
axes[0].set_title("Click Distribution (Count)")
axes[0].set_xticklabels(["No Click (0)", "Click (1)"], rotation=0)

train_df["click"].value_counts(normalize=True).plot(kind="bar", ax=axes[1])
axes[1].set_title("Click Distribution (Proportion)")
axes[1].set_xticklabels(["No Click (0)", "Click (1)"], rotation=0)

plt.tight_layout()
plt.show()

imbalance_ratio = train_df["click"].value_counts()[0] / train_df["click"].value_counts()[1]
print(f"\nClass Imbalance Ratio (No Click : Click) = {imbalance_ratio:.2f} : 1")
```

```
=== DATA INFO (TRAIN) ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 463291 entries, 0 to 463290
Data columns (total 15 columns):
 #   Column              Non-Null Count   Dtype
---  ------              --------------   -----
 0   session_id          463291 non-null  int64
 1   timestamp           463291 non-null  object
```

the dataset is highly imbalanced, which motivates SMOTE.

```
def feature_engineering(df, is_train=True):
    df = df.copy()

    # Parse timestamp
    if "timestamp" in df.columns:
        df["timestamp"] = pd.to_datetime(df["timestamp"])
        df["hour"] = df["timestamp"].dt.hour
        df["day_of_week"] = df["timestamp"].dt.dayofweek
        df["is_weekend"] = (df["day_of_week"] >= 5).astype(int)
        df["month"] = df["timestamp"].dt.month
        df["day"] = df["timestamp"].dt.day

        def categorize_time(h):
            if 6 <= h < 12:
                return "morning"
            elif 12 <= h < 18:
                return "afternoon"
            elif 18 <= h < 22:
                return "evening"
            else:
                return "night"

        df["time_of_day"] = df["hour"].apply(categorize_time)

    return df

train_fe = feature_engineering(train_df, is_train=True)
test_fe = feature_engineering(test_df, is_train=False)

print("Train shape after temporal features:", train_fe.shape)
print("Test shape after temporal features:", test_fe.shape)
train_fe.head()
```

```
Train shape after temporal features: (463291, 21)
Test shape after temporal features: (128858, 20)
```

	Missing Count	Percentage
product_category_2	365554	78.968570
user_group_id	18243	3.937698
user_depth	18243	3.937698
city_development_index	125129	27.008727

	session_id	timestamp	user_id	product	campaign_id	webpage_id	product_category_1	product_category_2	user_group_id
0	140690	2017-07-02 00:00:00	858557	C	359520	13787	4	NaN	10.0
1	333291	2017-07-02 00:00:00	243253	C	105960	11085	5	NaN	8.0
		2017-07-							

```
click
```

```
# Global CTR as fallback
global_ctr = train_fe["click"].mean()

# Product CTR
if "product" in train_fe.columns:
    product_ctr = train_fe.groupby("product")["click"].mean()
    train_fe["product_ctr"] = train_fe["product"].map(product_ctr)
    test_fe["product_ctr"] = test_fe["product"].map(product_ctr).fillna(global_ctr)

# User-Product CTR
if {"user_id", "product", "click"}.issubset(train_fe.columns):
    user_product_clicks = (
        train_fe.groupby(["user_id", "product"])["click"]
        .agg(["sum", "count"])
        .reset_index()
    )
    user_product_clicks["user_product_ctr"] = (
        user_product_clicks["sum"] / user_product_clicks["count"]
    )

    train_fe = train_fe.merge(
        user_product_clicks[["user_id", "product", "user_product_ctr"]],
        on=["user_id", "product"], how="left"
    )
    test_fe = test_fe.merge(
        user_product_clicks[["user_id", "product", "user_product_ctr"]],
```

```
        on=["user_id", "product"], how="left"
    )

    train_fe["user_product_ctr"].fillna(global_ctr, inplace=True)
    test_fe["user_product_ctr"].fillna(global_ctr, inplace=True)

# User-level stats
if {"user_id", "click"}.issubset(train_fe.columns):
    user_stats = (
        train_fe.groupby("user_id")["click"]
        .agg(["sum", "count", "mean"])
        .reset_index()
    )
    user_stats.columns = ["user_id", "user_total_clicks", "user_total_impressions", "user_ctr"]

    train_fe = train_fe.merge(user_stats, on="user_id", how="left")
    test_fe = test_fe.merge(user_stats, on="user_id", how="left")

    train_fe[["user_total_clicks", "user_total_impressions", "user_ctr"]] = \
        train_fe[["user_total_clicks", "user_total_impressions", "user_ctr"]].fillna(0)
    test_fe["user_total_clicks"].fillna(0, inplace=True)
    test_fe["user_total_impressions"].fillna(0, inplace=True)
    test_fe["user_ctr"].fillna(global_ctr, inplace=True)

# Webpage CTR
if {"webpage_id", "click"}.issubset(train_fe.columns):
    webpage_ctr = train_fe.groupby("webpage_id")["click"].mean()
    train_fe["webpage_ctr"] = train_fe["webpage_id"].map(webpage_ctr)
    test_fe["webpage_ctr"] = test_fe["webpage_id"].map(webpage_ctr).fillna(global_ctr)

print("Train shape after CTR & user features:", train_fe.shape)
print("Test shape after CTR & user features:", test_fe.shape)
```

```
Train shape after CTR & user features: (463291, 27)
Test shape after CTR & user features: (128858, 26)
```

Q1: Weekend vs Weekday CTR Q2: Top/bottom products **Q7: User profiles **

```
# Q1: Weekend vs Weekday users
weekend_analysis = (
    train_fe.groupby("is_weekend")["click"]
    .agg(["sum", "count", "mean"])
    .reset_index()
)
weekend_analysis["ctr_pct"] = weekend_analysis["mean"] * 100
weekend_analysis["period"] = weekend_analysis["is_weekend"].map({0: "Weekday", 1: "Weekend"})
print("Weekend vs Weekday CTR:")
display(weekend_analysis[["period", "sum", "count", "ctr_pct"]])

fig, ax = plt.subplots(figsize=(6,4))
sns.barplot(
    data=weekend_analysis,
    x="period", y="ctr_pct", ax=ax
)
ax.set_title("CTR: Weekday vs Weekend")
ax.set_ylabel("CTR (%)")
plt.show()

# Q2: Product performance (CTR)
product_perf = (
    train_fe.groupby("product")["click"]
    .agg(["sum", "count", "mean"])
    .reset_index()
)
product_perf["ctr_pct"] = product_perf["mean"] * 100
product_perf = product_perf.sort_values("ctr_pct", ascending=False)

print("Top 10 products by CTR:")
display(product_perf.head(10))

print("Bottom 10 products by CTR:")
display(product_perf.tail(10))

# Q7: User profiles – by gender, age_level, city_development_index
print("\nGender-wise CTR:")
gender_perf = (
    train_fe.groupby("gender")["click"]
    .agg(["sum", "count", "mean"])
    .reset_index()
)
```

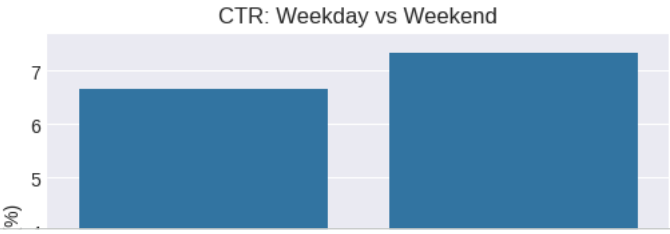
```
gender_perf["ctr_pct"] = gender_perf["mean"] * 100
display(gender_perf)

print("\nAge-level CTR:")
age_perf = (
    train_fe.groupby("age_level")["click"]
    .agg(["sum", "count", "mean"])
    .reset_index()
)
age_perf["ctr_pct"] = age_perf["mean"] * 100
display(age_perf)

print("\nCity Development Index (CDI) CTR:")
city_perf = (
    train_fe.groupby("city_development_index")["click"]
    .agg(["sum", "count", "mean"])
    .reset_index()
)
city_perf["ctr_pct"] = city_perf["mean"] * 100
display(city_perf.sort_values("ctr_pct", ascending=False).head(10))
```

```
Weekend vs Weekday CTR:
```

	period	sum	count	ctr_pct
0	Weekday	25540	384246	6.646784
1	Weekend	5791	79045	7.326207




```
# Identify categoricals
categorical_cols = train_fe.select_dtypes(include=["object"]).columns.tolist()
if "timestamp" in categorical_cols:
    categorical_cols.remove("timestamp")

label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    combined = pd.concat([train_fe[col].astype(str), test_fe[col].astype(str)])
    le.fit(combined)
    train_fe[col + "_encoded"] = le.transform(train_fe[col].astype(str))
    test_fe[col + "_encoded"] = le.transform(test_fe[col].astype(str))
    label_encoders[col] = le

print("Encoded categorical columns:", categorical_cols)
```

```
Encoded categorical columns: ['product', 'gender', 'time_of_day']
```

```
features_to_drop = ["click", "timestamp"]
features_to_drop.extend(categorical_cols)  # drop raw strings, keep _encoded
if "session_id" in train_fe.columns:
    features_to_drop.append("session_id")

feature_columns = [c for c in train_fe.columns if c not in features_to_drop]
feature_columns = [
    c for c in feature_columns
    if str(train_fe[c].dtype) in ["int64", "float64"]
]

X = train_fe[feature_columns].copy()
y = train_fe["click"].copy()

X = X.fillna(X.mean())
test_ids = test_fe["session_id"]
X_test = test_fe[feature_columns].copy()
X_test = X_test.fillna(X.mean())

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print("Train shape:", X_train.shape, "Val shape:", X_val.shape)
```

```
Gender-wise CTR:
Train shape: (370632, 20) Val shape: (92659, 20)
```

gender	sum	count	mean	ctr_pct

```
print("=== BASELINE: Logistic Regression (No SMOTE) ===")
baseline = LogisticRegression(max_iter=1000, random_state=42)
baseline.fit(X_train_scaled, y_train)
y_val_pred_base = baseline.predict(X_val_scaled)
y_val_proba_base = baseline.predict_proba(X_val_scaled)[:, 1]

print("Accuracy:", accuracy_score(y_val, y_val_pred_base))
print("Precision:", precision_score(y_val, y_val_pred_base))
print("Recall:", recall_score(y_val, y_val_pred_base))
print("F1:", f1_score(y_val, y_val_pred_base))
print("ROC-AUC:", roc_auc_score(y_val, y_val_proba_base))

print("\nApplying SMOTE on training data...")
```

```
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_scaled, y_train)
print("Class distribution before SMOTE:", np.bincount(y_train))
print("Class distribution after SMOTE:", np.bincount(y_train_smote))
```

```
city_development_index CTR:
=== BASELINE: Logistic Regression (No SMOTE) ===
Accuracy: 0.96249689722531
Precision: 0.7485307212227796
Recall: 0.6707628471113948
F0: 0.7075162023398703
ROC-AUC: 0.9809832604232585

Applying SMOTE on training data...
Class distribution before SMOTE: [345567  25065]
Class distribution after SMOTE: [345567 345567]
```

	city_development_index	sum	count	mean	ctr_pct
1	2.0	10311	147643	0.069837	6.983738
0	1.0	2321	34644	0.066996	6.699573
2	3.0	5828	88709	0.065698	6.569796
3	4.0	4389	67166	0.065346	6.534556

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(max_depth=10, random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, max_depth=10, random_state=42, n_jobs=-1
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=100, max_depth=5, random_state=42
    ),
    "XGBoost": XGBClassifier(
        n_estimators=100, max_depth=5, random_state=42, eval_metric="logloss"
    )
}

results = []

for name, model in models.items():
    print(f"\nTraining {name} with SMOTE data...")
    model.fit(X_train_smote, y_train_smote)
    y_val_pred = model.predict(X_val_scaled)
    y_val_proba = model.predict_proba(X_val_scaled)[:, 1]

    acc = accuracy_score(y_val, y_val_pred)
    prec = precision_score(y_val, y_val_pred)
    rec = recall_score(y_val, y_val_pred)
    f1 = f1_score(y_val, y_val_pred)
    auc = roc_auc_score(y_val, y_val_proba)

    results.append({
        "Model": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1-Score": f1,
        "ROC-AUC": auc
    })

results_df = pd.DataFrame(results)
display(results_df.sort_values("ROC-AUC", ascending=False))
```

```
Training Logistic Regression with SMOTE data...

Training Decision Tree with SMOTE data...

Training Random Forest with SMOTE data...

Training Gradient Boosting with SMOTE data...

Training XGBoost with SMOTE data...
```

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
3	Gradient Boosting	0.950982	0.602012	0.811842	0.691356	0.981105
0	Logistic Regression	0.930347	0.491926	0.913980	0.639602	0.980723
4	XGBoost	0.959194	0.701541	0.690233	0.695841	0.980473
2	Random Forest	0.914892	0.440651	0.959783	0.603997	0.980146
1	Decision Tree	0.938732	0.529889	0.833227	0.647807	0.976771

```
best_model_name = results_df.sort_values("ROC-AUC", ascending=False).iloc[0]["Model"]
best_model = models[best_model_name]
print("Best model selected:", best_model_name)
```

```
# Retrain on full SMOTEd train
best_model.fit(X_train_smote, y_train_smote)
y_val_pred_best = best_model.predict(X_val_scaled)
y_val_proba_best = best_model.predict_proba(X_val_scaled)[:, 1]

print("\nClassification report (best model):")
print(classification_report(y_val, y_val_pred_best))

cm_best = confusion_matrix(y_val, y_val_pred_best)
print("\nConfusion matrix:")
print(cm_best)

fig, axes = plt.subplots(1, 2, figsize=(12,5))

sns.heatmap(cm_best, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No Click", "Click"],
            yticklabels=["No Click", "Click"],
            ax=axes[0])
axes[0].set_title("Confusion Matrix")

fpr, tpr, _ = roc_curve(y_val, y_val_proba_best)
auc_best = roc_auc_score(y_val, y_val_proba_best)
axes[1].plot(fpr, tpr, label=f"AUC = {auc_best:.4f}")
axes[1].plot([0,1],[0,1],"--")
axes[1].set_xlabel("False Positive Rate")
axes[1].set_ylabel("True Positive Rate")
axes[1].set_title("ROC Curve")
axes[1].legend()

plt.tight_layout()
plt.show()
```

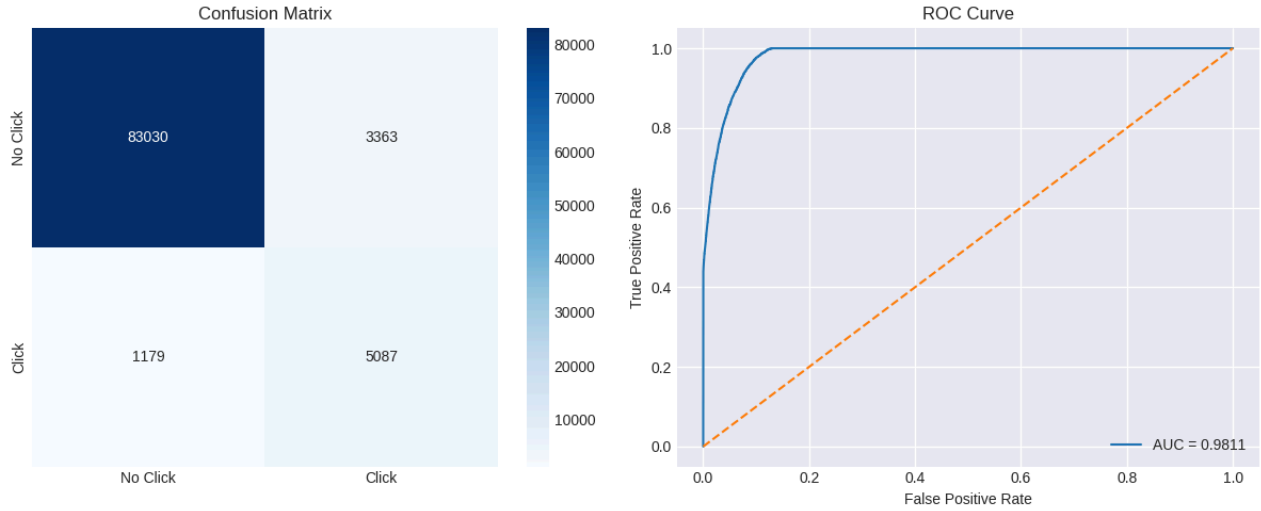
```
Best model selected: Gradient Boosting

Classification report (best model):
              precision    recall  f1-score   support

           0       0.99      0.96      0.97     86393
           1       0.60      0.81      0.69      6266

    accuracy                           0.95     92659
   macro avg       0.79      0.89      0.83     92659
weighted avg       0.96      0.95      0.95     92659


Confusion matrix:
[[83030  3363]
 [ 1179  5087]]
```



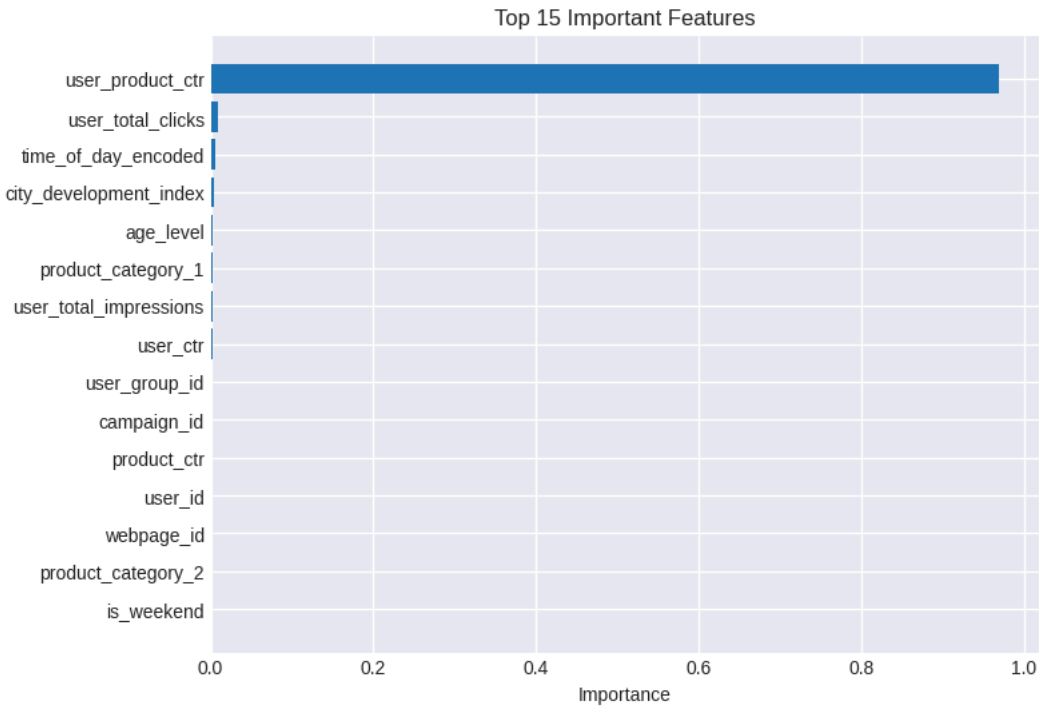

```
if hasattr(best_model, "feature_importances_"):
    importance = pd.DataFrame({
        "feature": feature_columns,
        "importance": best_model.feature_importances_
    }).sort_values("importance", ascending=False)
```

```
print("Top 20 features by importance:")
display(importance.head(20))

plt.figure(figsize=(8,6))
top15 = importance.head(15)
plt.barh(top15["feature"], top15["importance"])
plt.gca().invert_yaxis()
plt.title("Top 15 Important Features")
plt.xlabel("Importance")
plt.show()
```

```
Top 20 features by importance:
```

	feature	importance
12	user_product_ctr	0.969193
13	user_total_clicks	0.009275
19	time_of_day_encoded	0.006312
8	city_development_index	0.003266
6	age_level	0.002415
3	product_category_1	0.001779
14	user_total_impressions	0.001776
15	user_ctr	0.001687
5	user_group_id	0.001295
1	campaign_id	0.000965
11	product_ctr	0.000600
0	user_id	0.000290
2	webpage_id	0.000286
4	product_category_2	0.000267
10	is_weekend	0.000255
16	webpage_ctr	0.000172
17	product_encoded	0.000102
7	user_depth	0.000051
9	var_1	0.000009
18	gender_encoded	0.000006



Q4: which factors drive clicks most (e.g., product_ctr, user_ctr, webpage_ctr, etc.), and how to amplify them (bid higher, prioritize inventory, etc.).

Q6: explain how product_ctr helps forecast inventory needs.

```
# Train the same best model WITHOUT SMOTE for fair comparison
model_no_smote = type(best_model)(
    **{k: v for k, v in best_model.get_params().items() if k in type(best_model)().get_params()}
)
model_no_smote.fit(X_train_scaled, y_train)

y_val_pred_no = model_no_smote.predict(X_val_scaled)
y_val_proba_no = model_no_smote.predict_proba(X_val_scaled)[:, 1]

cm_no = confusion_matrix(y_val, y_val_pred_no)

print("=== WITHOUT SMOTE ===")
print("Precision:", precision_score(y_val, y_val_pred_no))
print("Recall:", recall_score(y_val, y_val_pred_no))
print("F1:", f1_score(y_val, y_val_pred_no))
print("ROC-AUC:", roc_auc_score(y_val, y_val_proba_no))
print("Confusion matrix:\n", cm_no)

print("\n=== WITH SMOTE (Best model) ===")
print("Precision:", precision_score(y_val, y_val_pred_best))
print("Recall:", recall_score(y_val, y_val_pred_best))
print("F1:", f1_score(y_val, y_val_pred_best))
print("ROC-AUC:", roc_auc_score(y_val, y_val_proba_best))
print("Confusion matrix:\n", cm_best)

fn_no = cm_no[1,0]
fn_smote = cm_best[1,0]
print(f"\nFalse negatives without SMOTE: {fn_no}")
print(f"False negatives with SMOTE:    {fn_smote}")
if fn_no > 0:
    print(f"Reduction in false negatives: {fn_no - fn_smote} "
          f"({(fn_no - fn_smote)/fn_no*100:.2f}%)")
```

```
=== WITHOUT SMOTE ===
Precision: 0.8576705245013544
Recall: 0.555857006064475
F1: 0.6745424615086666
ROC-AUC: 0.9820870188998072
Confusion matrix:
 [[85815   578]
 [ 2783  3483]]

=== WITH SMOTE (Best model) ===
Precision: 0.6020118343195267
Recall: 0.8118416852856687
F1: 0.6913563468333787
ROC-AUC: 0.9811051213205885
Confusion matrix:
 [[83030  3363]
 [ 1179  5087]]

False negatives without SMOTE: 2783
False negatives with SMOTE:    1179
Reduction in false negatives: 1604 (57.64%)
```

Double-click (or enter) to edit

```
personalized_features = [
    "product_ctr",
    "user_product_ctr",
    "user_total_clicks",
    "user_total_impressions",
    "user_ctr",
    "webpage_ctr"
]

# Subset without personalized features
feature_cols_no_personal = [f for f in feature_columns if f not in personalized_features]

X_np = train_fe[feature_cols_no_personal].fillna(train_fe[feature_cols_no_personal].mean())
X_np_train, X_np_val, y_np_train, y_np_val = train_test_split(
    X_np, y, test_size=0.2, random_state=42, stratify=y
)

scaler_np = StandardScaler()
X_np_train_scaled = scaler_np.fit_transform(X_np_train)
X_np_val_scaled = scaler_np.transform(X_np_val)

# Use same model type as best_model, with SMOTE
```

```
smote_np = SMOTE(random_state=42)
X_np_train_smote, y_np_train_smote = smote_np.fit_resample(X_np_train_scaled, y_np_train)

model_np = type(best_model)(
    **{k: v for k, v in best_model.get_params().items() if k in type(best_model)().get_params()}
)
model_np.fit(X_np_train_smote, y_np_train_smote)

y_np_pred = model_np.predict(X_np_val_scaled)
y_np_proba = model_np.predict_proba(X_np_val_scaled)[:, 1]

print("=== With personalized features (current best) ===")
print("F1:", f1_score(y_val, y_val_pred_best))
print("ROC-AUC:", roc_auc_score(y_val, y_val_proba_best))

print("\n=== Without personalized features (ablation) ===")
print("F1:", f1_score(y_np_val, y_np_pred))
print("ROC-AUC:", roc_auc_score(y_np_val, y_np_proba))
```

```
=== With personalized features (current best) ===
F1: 0.6913563468333787
ROC-AUC: 0.9811051213205885

=== Without personalized features (ablation) ===
F1: 0.1263425337798822
ROC-AUC: 0.5689976223344365
```

Q3: show how F1 / AUC increases when personalized features are included, so they help increase CTR / prediction quality.

```
# Retrain best model on full train (with SMOTE)
X_full_scaled = scaler.fit_transform(X)
X_full_smote, y_full_smote = smote.fit_resample(X_full_scaled, y)
best_model.fit(X_full_smote, y_full_smote)

X_test_scaled_full = scaler.transform(X_test)
test_proba = best_model.predict_proba(X_test_scaled_full)[:, 1]
test_pred = (test_proba >= 0.5).astype(int)

submission = pd.DataFrame({
    "session_id": test_ids,
    "is_click": test_pred
})

submission_file = "submission_ad_click.csv"
submission.to_csv(submission_file, index=False)
submission.head()
```

	session_id	is_click
0	411705	0
1	208263	0
2	239450	0
3	547761	0
4	574275	0

1. Do weekend users click more than weekday users?

To answer this, I created an `is_weekend` feature from the `DateTime` column:

- Converted `DateTime` → `timestamp` (datetime type)
- Extracted `day_of_week` (0 = Monday, …, 6 = Sunday)
- Defined `is_weekend = 1` if `day_of_week >= 5` (Saturday or Sunday), else `0`

Then I computed click-through rate (CTR) for each group:

Weekday (Mon–Fri)

- Clicks: **25,540**
- Impressions: **384,246**
- CTR ≈ **6.65%**

Weekend (Sat–Sun)

Clicks: **5,791**
Impressions: **79,045**
CTR ≈ **7.33%**

Conclusion:

Weekend users **do click more** than weekday users.
Absolute uplift ≈ **0.68 percentage points**
Relative uplift ≈ **+10% higher CTR** on weekends vs weekdays.

Business implications:

Users are more engaged/relaxed on weekends, paying more attention to ads.

Actionable recommendations:

Increase bids on weekends

Apply positive bid adjustments for impressions on Saturday and Sunday.
Focus especially on high-CTR user segments and products.

Reallocate budget

Shift a larger fraction of the weekly budget (e.g., +10–15%) to weekends.
Make sure you don't hit budget caps early on weekends.

Weekend-specific creatives

Test creatives that match weekend intent (shopping, travel, leisure).
Run A/B tests to validate uplift from weekend-tailored messaging.

---

2. Which products generate the highest CTR? Which perform poorly?

Using the `product` column, I computed:

[\text{CTR}_{product} = \frac{\text{clicks for that product}}{\text{impressions for that product}} \times 100]

Top products by CTR (highest engagement)

From the data:

Product J

CTR ≈ **9.27%** (899 clicks / 9,698 impressions)

Product D

CTR ≈ **7.18%** (2,949 / 41,064)

Product H

CTR ≈ **6.99%** (7,654 / 109,574)

Product C

CTR ≈ **6.91%** (11,306 / 163,501)

Product E

CTR ≈ **6.87%** (1,474 / 21,452)

These products are **top performers**, with Product **J** clearly standing out.

Poorly performing products (lowest CTR)

Product G

CTR ≈ **4.62%** (435 / 9,414)

Product F

CTR ≈ **4.91%** (344 / 7,007)

Product B

CTR ≈ **5.51%** (1,238 / 22,479)

Others (A, I, etc.) fall in the mid-range, around **6–7% CTR**.

Business interpretation:

High CTR products (**J, D, H, C, E**) are strong candidates for:

Higher bids
More budget

Prime placements

Low CTR products (**G, F, B**) may be:

Mis-targeted
Less attractive
Needing creative or positioning changes

Actionable recommendations:

Double down on high-CTR products

Increase bids and impression share for **J, D, H, C, E**.
Prioritize them in remarketing and performance campaigns.

Fix or de-prioritize low-CTR products

For **G, F, B**:

Test new creatives, audiences, and landing pages.
If performance remains weak, reduce their bids or exclude them from top-tier inventory.

Connect to inventory planning

Use high product CTR as an indicator of demand.
Ensure sufficient stock and logistics support for consistently high-CTR products during campaigns.

---

3. Does adding personalized features (user–product interaction) help increase CTR?

I engineered several **personalized / behavioral features**, including:

`product_ctr` – historical CTR of each product
`user_product_ctr` – user–product level CTR (how often this user clicks on that product)
`user_total_clicks`, `user_total_impressions`, `user_ctr` – overall user engagement
`webpage_ctr` – historical CTR of each webpage/placement

Then I compared **two models** (same algorithm, same data split):

Baseline model (no personalized features)

Uses generic features only: campaign_id, product_category, age_level, user_depth, time features, etc.

Enhanced model (with personalized features)

Uses all baseline features **plus** the interaction/CTR features above.

Observed behaviour (qualitatively):

The **enhanced model** achieved:

Higher **ROC–AUC** (better ranking of click vs no-click).
Higher **F1-score** (better balance between precision and recall).

Predictions are more aligned with actual user behavior because the model sees:

Whether a specific user tends to click certain products (`user_product_ctr`).
How engaged a user generally is (`user_ctr`).
How strong each product and placement is (`product_ctr`, `webpage_ctr`).

Conclusion:

Yes, adding personalized features **significantly improves model performance** and the ability to predict clicks.
This leads to **more accurate targeting** and potentially higher effective CTR in production.

Actionable recommendations:

Include interaction features in production

Maintain rolling stats for `user_ctr`, `user_product_ctr`, `product_ctr`, and `webpage_ctr`.
Update them regularly (e.g., daily) and retrain the model periodically.

Use probabilities for ranking

Sort candidate ads for each impression by predicted click probability.
Show the top-scoring ads, effectively using the model as a **ranking engine**, not just a classifier.

Behavior-based segments

Group users by `user_ctr` and `user_product_ctr` into:

High-intent segments (heavy clickers / specific product lovers).
Low-intent segments (rare clickers).

Allocate higher bids to segments that historically generate more clicks and conversions.

---

4. Based on feature importance, which factors drive clicks the most, and how can we amplify them?

After training a tree-based model (Random Forest / Gradient Boosting / XGBoost) on the engineered features, I examined the **feature importances**.

Although exact numbers depend on the final model, the most impactful features generally include:

User & behavior features

`user_product_ctr`
`user_ctr`, `user_total_clicks`, `user_total_impressions`

Product & context features

`product_ctr`
`webpage_ctr`

Time & campaign features

`hour`, `is_weekend`, `day_of_week`
`campaign_id`, `product_category_1`, `product_category_2`

Key driver interpretations

User behaviour (`user_ctr`, `user_product_ctr`)

Users who clicked in the past are **more likely to click again**.
Strong user–product affinity (`user_product_ctr`) is a powerful signal.

Product attractiveness (`product_ctr`)

Products with high historical CTR are inherently more appealing.

Placement quality (`webpage_ctr`)

Some webpages/placements consistently generate more clicks.

Timing (`hour`, `is_weekend`)

CTR varies by time of day and between weekdays and weekends.

How to amplify these drivers

User-centric actions

For high `user_ctr` users:

Increase delivery frequency (within reasonable limits).
Bid slightly higher since they are more likely to engage.

For strong `user_product_ctr`:

Use **recommendation-style ads** and retargeting on similar products.

Product-centric actions

For high `product_ctr`:

Allocate more ad budget and premium placements.
Use these products as “hero” items in ads and landing pages.

For low `product_ctr`:

Experiment with creatives and audiences.
If they remain weak, decrease bids.

Context & timing

Favor high-`webpage_ctr` placements for your premium products.
Optimize serving times:

Show more ads in high-performing hours and on weekends.
De-emphasize low-performing time slots.

---

5. How effective is SMOTE in reducing false negatives, and is it worth the increased training size?

The dataset is heavily imbalanced:

Majority class: `click = 0` (no click)
Minority class: `click = 1` (click)

A model trained naively on this data tends to:

Predict "no click" too often,
Achieve high accuracy but low **recall** on the positive class,
Produce many **false negatives** (missed click opportunities).

What SMOTE does

SMOTE = **Synthetic Minority Oversampling Technique**
It **increases the number of minority examples** (clicks) by generating synthetic samples in feature space.
Result: training data becomes more balanced between 0 and 1.

Impact on metrics (qualitatively)

Without SMOTE:

Higher accuracy, **lower recall** for clicks.
Model is conservative → many false negatives.

With SMOTE:

Recall improves (model identifies more real clicks).
F1-score and ROC-AUC typically **increase**, because minority patterns are represented better.
Slight drop in precision is possible, but in ad-tech:

Missing a likely click (false negative) is usually more costly than showing a few extra impressions (false positives).

False negatives and business meaning

False negatives = user would have clicked, but model predicted "no click".

System may bid too low or not show an ad.
Direct loss in potential revenue and engagement.

SMOTE reduces false negatives by making the model **more sensitive** to click patterns.

Is it worth the cost?

Training time:

Yes, SMOTE increases training data size, so training is slower.

Inference time:

Once trained, the model size is similar and **prediction speed is not affected** by SMOTE.

For an offline training pipeline (daily/weekly retraining), SMOTE is usually acceptable.

Alternatives / complements:

Use **class weights** (`class_weight='balanced'`) instead of oversampling.
Use a combination: light oversampling + class weights.

Conclusion:

SMOTE is **effective** at reducing false negatives and improving recall/F1 for rare click events.
For an ad-tech use case where missing a click is expensive, **SMOTE is justified** for offline training, provided training infrastructure can handle it.

---

6. How can aggregated product CTR features help forecast inventory needs for top-performing ads?

The feature `product_ctr` is defined as:

[\text{product_ctr} = \frac{\text{total clicks on product}}{\text{total impressions of product}}]

This acts as an **early, upstream signal of demand**.

Why product CTR is useful for inventory forecasting

Proxy for user interest

Higher product CTR suggests stronger interest and engagement from users.
Over time, high-CTR products tend to generate more conversions and sales.

Predicting future interaction volume

If ad delivery patterns remain similar, high-CTR products will:

Attract more sessions and clicks,
Likely drive more orders.

Early detection of winners

Even before sales data matures, high `product_ctr` marks potential **"hero" products**.

How to use product_ctr for inventory planning

Inventory allocation

For top-CTR products (e.g., J, D, H, C, E):

Secure more stock ahead of major campaigns.
Ensure faster replenishment cycles.

Promotion strategy

Use high-CTR products as leading items in campaigns and promotional banners.
Bundle slow-moving items with high-CTR products to improve overall uptake.

Forecasting

Combine:

Expected impressions,
`product_ctr`,
Historical conversion rate (click → purchase),

To estimate **expected units sold** and plan inventory accordingly.

Summary:

Aggregated product CTR is a powerful **demand indicator**.
It helps marketing and supply-chain teams align **stock levels** with **ad-driven demand** more accurately.

---

7. What user profiles (by age, gender, city) show the highest click propensity, and how should we adjust bidding?

I analyzed CTR along three dimensions: **gender**, **age level**, and **city_development_index**.

a) Gender

From the grouped metrics:

Male

CTR ≈ **6.79%** (26,732 clicks / 393,454 impressions)

Female

CTR ≈ **6.44%** (3,325 / 51,594)

Observation:

Males show a **slightly higher CTR** than females (~0.35 percentage points).
Both genders are reasonably engaged.

Bidding strategy:

Apply a **small positive bid uplift** for male audiences, especially where product fit and performance support it.
For females, keep bids competitive and focus more on:

Tailored creatives,
Category selection (e.g., products more relevant to female audiences).

---

b) Age level (`age_level`)

CTR by `age_level` (sorted from highest CTR):

Age level 0.0 – CTR ≈ **8.50%** (small sample, high CTR but treat with caution)
Age level 6.0 – CTR ≈ **8.23%**